

Case Study:

Encois

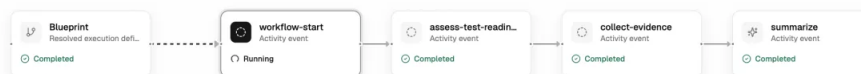
Designing evidence-backed
organizational intelligence

Workflow canvas

Topology is derived from emitted step events. It updates as execution reports activity.

Updates stopped after terminal state

• Run topology Polled 9:36:22 AM
○ Running ○ Waiting △ Failed ○ Pending



01

Google Cloud architecture

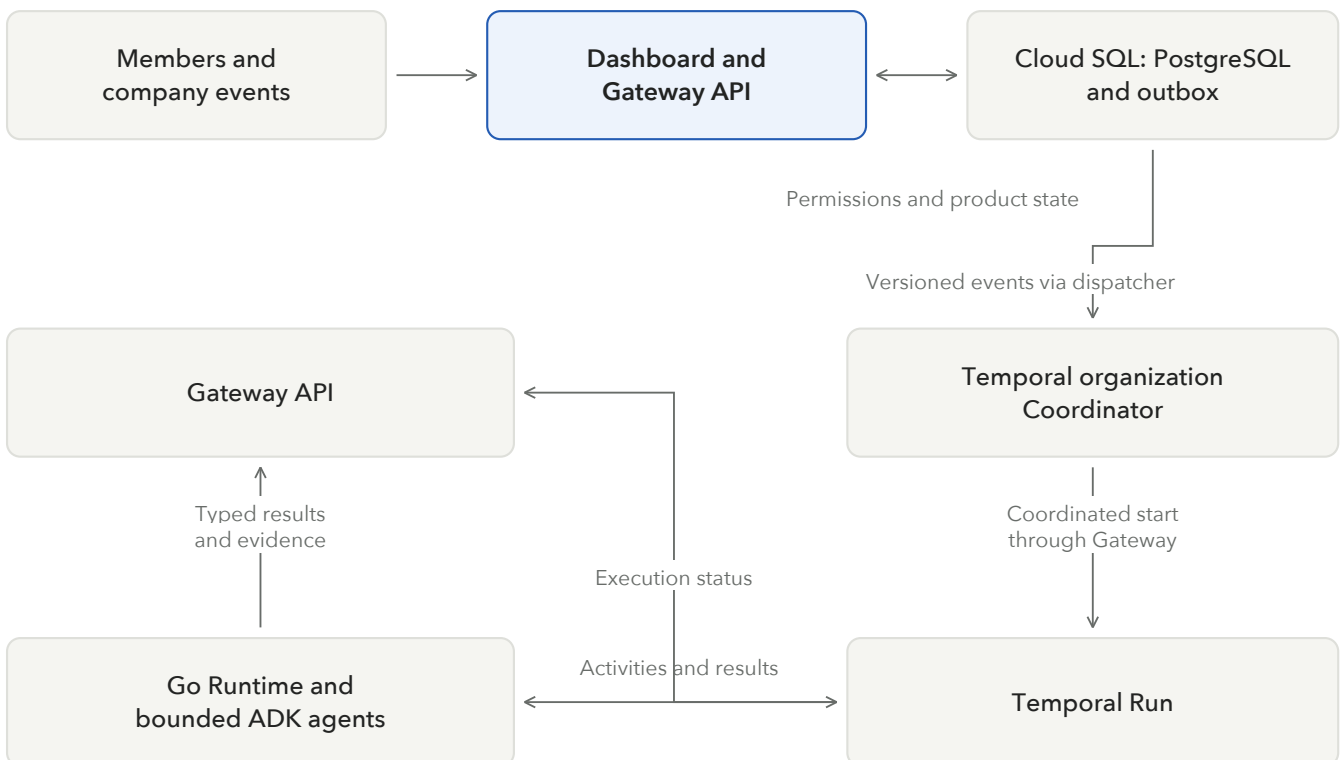
Encois is an organizational intelligence system designed to explain what changed across company tools, why it matters and what deserves attention. I designed its architecture around scoped evidence and durable investigations: a team can ask whether a release is safe, follow the reasoning back to its sources and retain the context of the investigation as new events arrive. The product brings operational signals together while keeping permissions, execution and AI interpretation under separate, explicit control.

The business problem

Release decisions depend on information spread across issues, pull requests, checks and documents. Encois connects those signals into an investigation with source references and access rules. The team can see what is blocked, inspect the evidence and decide what to do next.

The system separates product control, workflow execution and data access:

- **Dashboard and Gateway API:** accept requests, check membership and scope, and manage Sources, workflow definitions and Run records.
- **Temporal and the Go Runtime:** coordinate work and execute the approved steps. Google ADK agents handle reasoning inside those steps.
- **Private Agent Gateway:** validate tool requests, read permitted provider data and return typed evidence.



The Gateway records accepted work. Temporal keeps execution history. The Runtime executes steps and returns results.

Services and data responsibilities

Google Cloud services and their roles

Service	Responsibility	Main access path
Cloud Run	Separate Dashboard, API, Runtime and private Agent Gateway services	Public requests enter the API; tool access uses the private gateway
Cloud SQL for PostgreSQL	Permissions, configuration, Run records, evidence references and outbox	Gateway transactions and product queries
Cloud Storage	Original Source artifacts	Scoped reads through the Agent Gateway
Spanner Graph	Structured organizational facts and relationships	Evidence-linked queries through the Agent Gateway
Memory Bank	Agent-specific semantic context across Workflows	Runtime retrieval and context distillation
Vertex AI and Gemini	Reasoning and synthesis	Validated, scoped inputs from the agent Runtime
Secret Manager	Provider credentials	Backend resolution when an integration needs access
Cloud Logging and Cloud Trace	Errors, timing, retries and request traces	Correlated telemetry across services

PostgreSQL stores product records and references. Source content stays in Cloud Storage and is read through the private gateway. Temporal receives artifact references and execution context, so its history does not need to contain raw documents or credentials.

The memory layers have different jobs. Spanner Graph represents facts and their relationships. Memory Bank supplies semantic context for agents. Both derive context from Sources; the original revision remains the evidence behind it. Temporal separately owns execution history, waits and recovery.

03

Source revisions and workflow definitions

Two versioned models keep each result tied to its inputs and execution rules.

- **Integration and Source:** an Integration connects an organization to a provider. A Source selects a permitted resource, such as a repository or Jira project, within that connection.
- **Source Revision:** an immutable observation of that resource. Webhooks and ingestion register new revisions as provider data changes.
- **Template and Blueprint:** a reviewed Template supplies a starting pattern. The approved Blueprint defines steps, dependencies, conditions, timers and approval points.
- **Workflow and Run:** a Workflow is a reusable process. A Run is one execution bound to an approved Blueprint snapshot. Later edits do not rewrite that Run.

The Agent Gateway reads source artifacts and extracts facts. Evidence keeps the provider reference, observation and ingestion times, freshness, transformation version and visibility scope where available. Graph and memory projections use that evidence while preserving the reference back to its source.

Sun Inc > Organization > Sources > Source details

10

John Smith

Architecture Decisions

← All sources

application/json · active

Freshness

Fresh

Expires 10/23/2026, 8:59:18 AM

Latest revision

r1

9/23/2026, 9:29:18 AM

Observed

9/23/2026, 8:29:18 AM

Provider observation time

Ingested

9/23/2026, 8:59:18 AM

Normalized into context

Scope and provenance

The workspace keeps access scope separate from provider configuration and immutable revision content.

Provider

Integration

Read scope

Visibility scope

Uploaded document

Not applicable

b71bfff87-38e3-4d48-b527-c7ec8441f37d

b71bfff87-38e3-4d48-b527-c7ec8441f37d

Revisions

Immutable snapshots retained with provenance metadata.

r1

application/json · observed 9/23/2026, 8:29:18 AM · ingested 9/23/2026, ...

active

Ingest

Ingestion activity

Processing activity for this source.

completed

graph_projected · 14 facts

bootstrap

Register a new revision

Add a provider-backed immutable snapshot. Existing revisions are never overwritten.

Source details connect the revision, scope and freshness to its ingestion history.

SYSTEM DESIGN

04 / 08

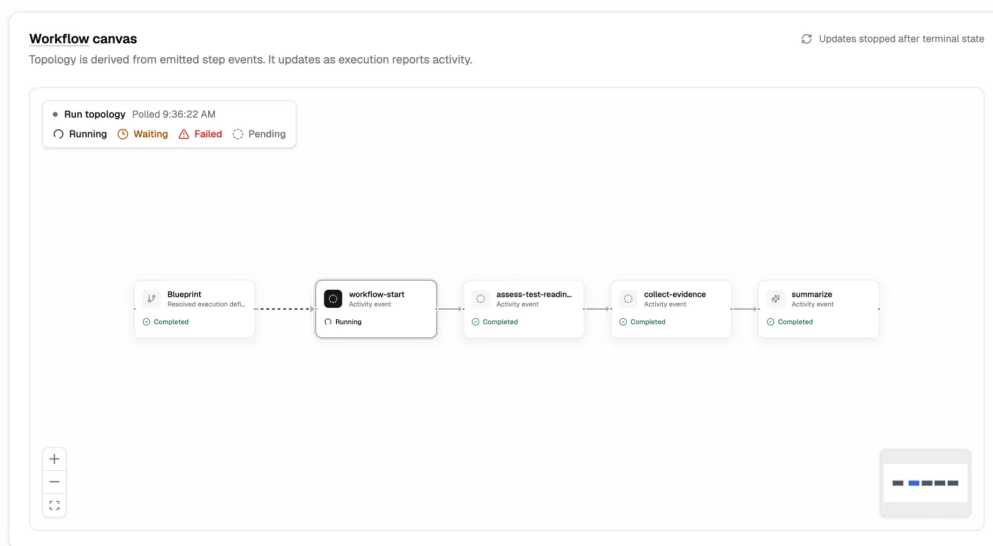
Retries, waits and recovery

The design gives command delivery and workflow execution separate recovery rules.

- **Atomic command recording.** The Gateway saves product changes and an outbox event in one PostgreSQL transaction. A saved Run therefore has a recorded command that the dispatcher can deliver.
- **Delivery retries.** The dispatcher leases an outbox row and sends its event to the organization's Coordinator. Delivery is at least once. Stable event and Workflow identities let the receiver deduplicate repeated delivery.
- **Organization coordination.** Each organization has one long-lived Coordinator Workflow. Signals carry lifecycle events. Continue-As-New keeps coordination going without accumulating one unbounded execution history.
- **Step execution.** The Go Runtime runs the approved Blueprint. Independent ready steps can run in parallel. An ADK agent runs as a Temporal Activity; model and tool calls stay outside deterministic Workflow code.
- **Execution limits.** Agent definitions include tool allowlists, a timeout, retry limit and budget. Temporal retries the Activity as a unit. Internal model turns therefore have coarser retry and execution visibility.
- **Wait, pause and cancel.** Timers, external conditions and approval Signals can leave a Run waiting without losing its place. An explicit pause is a separate versioned product command. Temporal also owns cancellation and execution recovery.

These controls serve different purposes. Blueprint dependencies determine when work may start. Timeouts and retry limits bound execution. Signals supply input to work that is already in progress.

The API exposes that distinction in Run status. An accepted Run appears as **preparing** before Temporal starts it. Temporal then supplies execution status; PostgreSQL keeps the product-facing projection. If a service is unavailable, the API reports stale or unavailable state.

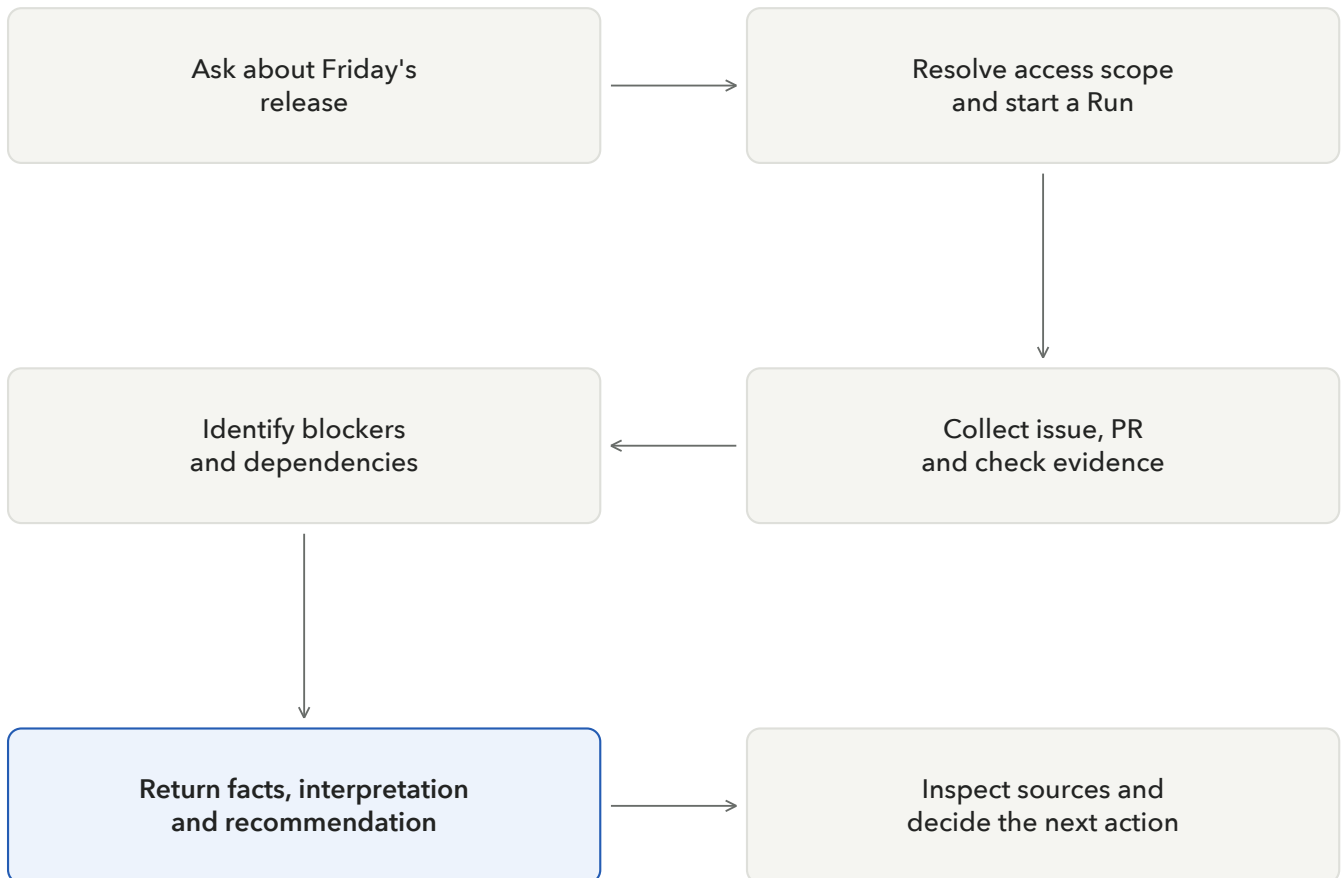


The Run view shows the approved Blueprint and which steps are pending, running or complete.

05

A release investigation

A delivery manager asks whether Friday's release is safe. The investigation uses permitted Sources and an approved Blueprint.



The system gathers and explains evidence. The person makes the release decision.

The result separates three things:

- **Facts:** what the permitted Sources returned, with references and freshness.
- **Interpretation:** which blockers or dependencies connect those facts.
- **Recommendation:** what the person should review or decide next.

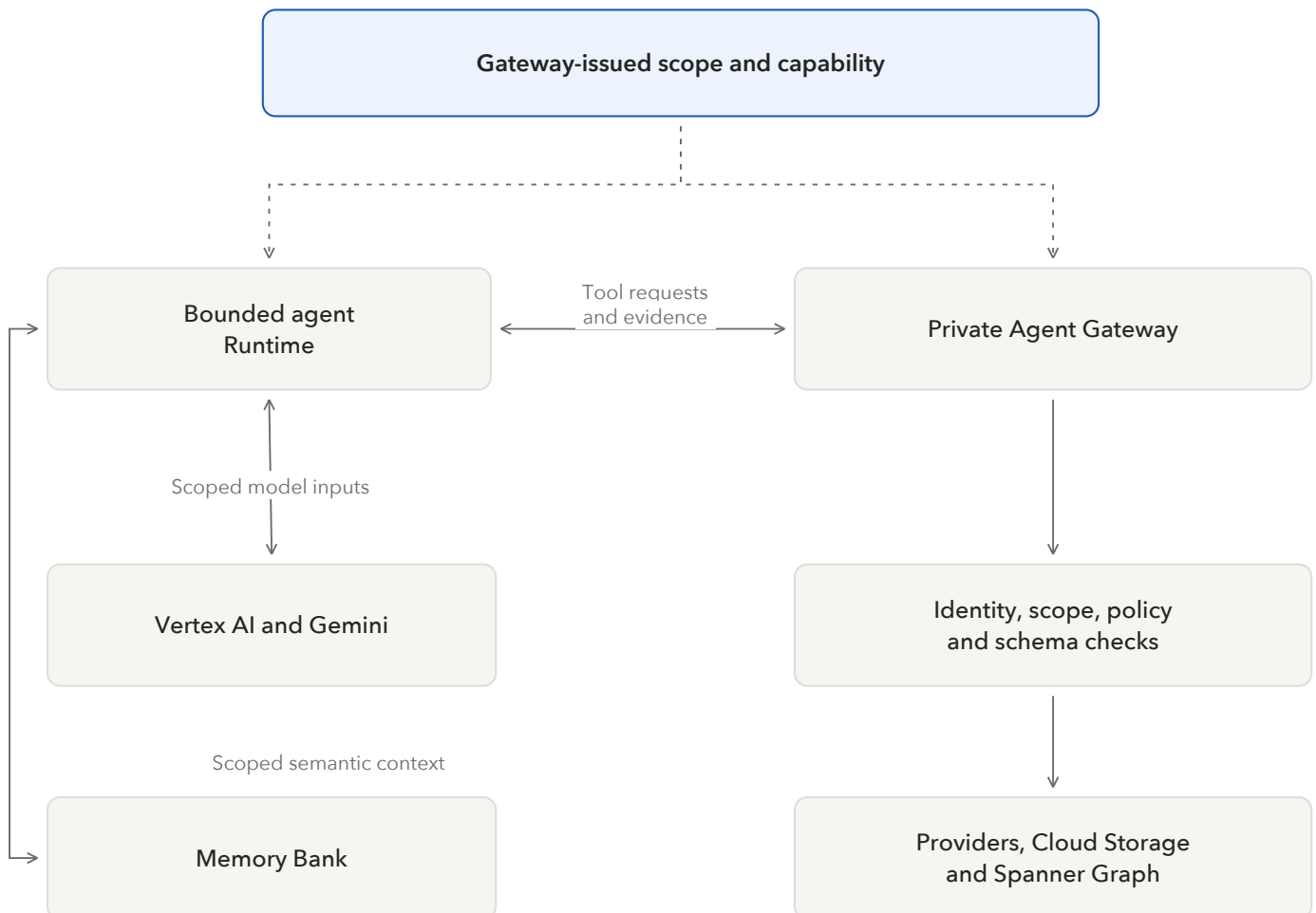
Tool access is read-only by default. A recommendation does not trigger a change in Jira, GitHub or another provider.

Authorization beyond the request

Organization is the tenant boundary. Departments, teams and projects narrow access inside it. The Gateway resolves effective scope from the authenticated member and selected unit before starting work.

That scope travels with the Run because execution can outlive the original request. The checks happen at several boundaries:

- **Gateway API:** establish the actor, organization and authorized unit IDs.
- **Runtime:** verify execution context and apply the agent's allowed tools and resource limits.
- **Private Agent Gateway:** check service identity, the short-lived signed capability, actor, scope, policy version, tool name and arguments.
- **Data adapters:** enforce the supplied scope when reading artifacts, organizational facts or semantic context.



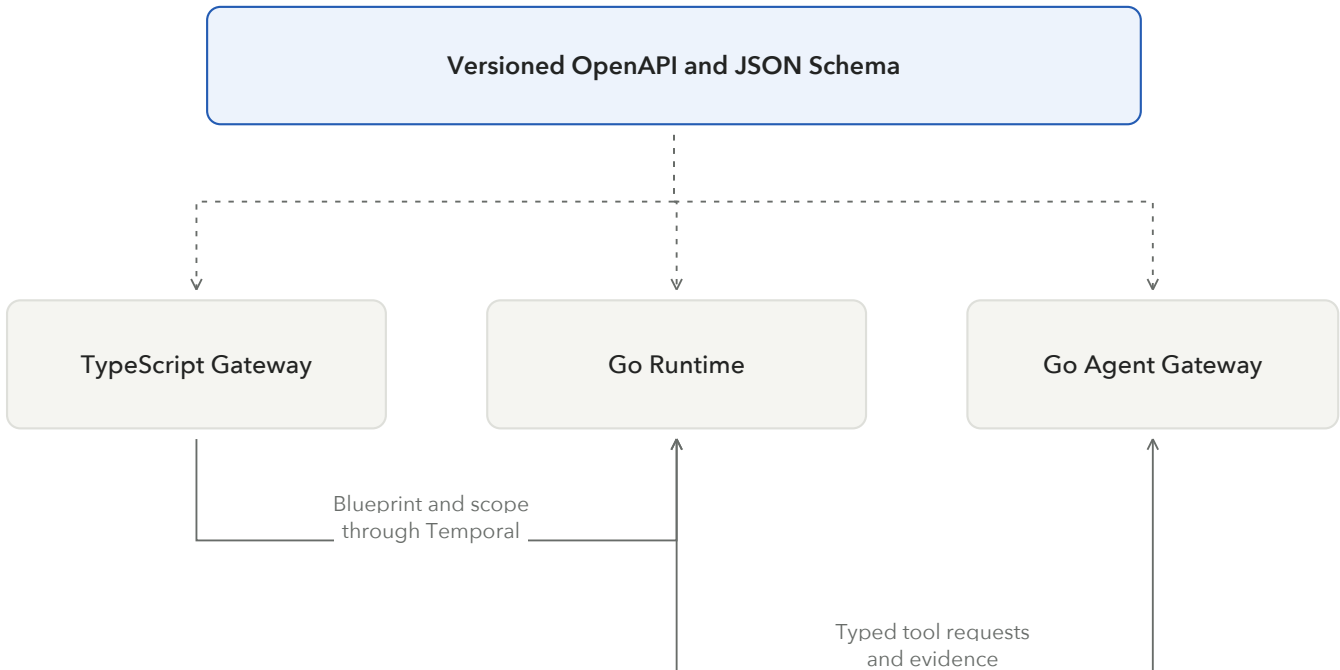
The same authorization context follows execution and data access. Model-generated identifiers cannot grant permission.

Provider tokens are resolved on the backend and stay outside the browser, model context and Temporal history. Tool arguments and provider content remain untrusted inputs. Missing scope or an unknown contract version stops the request; provider authentication failure stays visible as a need to reauthorize.

07

Shared contracts and tracing

The public Gateway uses TypeScript. The Runtime and private Agent Gateway use Go. They share versioned OpenAPI and JSON Schema definitions for messages, rather than relying on shared database models.



Dotted arrows show shared definitions. Solid arrows show messages exchanged between services. Each boundary validates its inputs.

- **Versioned payloads:** Coordinator events, Workflow inputs, tool manifests, evidence records and private service requests have explicit schemas.
- **Complete execution inputs:** the Runtime receives the approved Blueprint and context through Temporal. It does not query PostgreSQL to reconstruct missing application state.
- **Provider adapters:** provider-specific payloads stop at the Agent Gateway and become typed evidence records for the rest of the system.
- **Correlated traces:** request, organization, Workflow and Run IDs connect API calls, outbox delivery, Activities and tool requests.
- **Focused telemetry:** logs record duration, retries, provider, model and error class without storing raw credentials or unrestricted documents.

Summary

Encois separates product state, workflow execution, evidence, memory and model reasoning. Google Cloud services give each data type a clear home. Temporal handles long-running work, and scoped contracts keep access rules attached to it. The user gets a clear result with sources to inspect and a decision to make.